

植物养护模拟系统需求文档

1. 系统概述

植物养护模拟系统是一个基于LVGL图形库的嵌入式应用，模拟多种植物（向日葵、番茄、仙人掌、食人花）在不同环境条件下的生长过程。系统通过实时环境模拟、植物状态更新和用户交互，提供沉浸式的植物养护体验。

2. 核心功能需求

2.1 环境模拟系统

- 动态环境生成：
 - 随机生成温度(0-40°C)、湿度(0-100%)、光照强度(0-100%)
 - 模拟季节变化（春夏秋冬）
 - 实现多种天气类型（晴朗、多云、下雨、下雪、大风）
 - 天气和季节因素影响环境参数
- 环境更新机制：
 - 每2秒模拟一天的环境变化
 - 环境参数持久化存储

2.2 植物生长模拟

- 植物类型支持：
 - 向日葵、番茄、仙人掌、食人花
- 生长阶段管理：
 - 完整生命周期：种子→发芽→幼苗→成熟→开花→结果
 - 各阶段有不同生长时长需求
- 植物状态属性：
 - 健康值(0-100)
 - 年龄(天数)
 - 生长进度(0-100%)
 - 需求状态(需浇水/需施肥/需除虫)
- 环境互动机制：
 - 植物健康受环境参数影响
 - 每种植物有特定的环境偏好

2.3 用户管理系统

- 用户属性：
 - 金币管理（上限99999）
 - 植物拥有状态
 - 植物类型记录

- **经济系统：**
 - 每日自动获得50金币
 - 植物商店系统

2.4 图形用户界面

- **植物展示：**
 - 根据生长阶段显示不同植物图像
 - 实时显示健康状态和生长阶段
- **环境参数显示：**
 - 温度、湿度、光照强度
 - 当前天气和季节
- **用户界面元素：**
 - 金币显示
 - 植物状态标签
 - 环境参数标签

2.5 数据持久化

- **自动保存机制：**
 - 程序正常退出时保存
 - 接收SIGINT/SIGTERM信号时保存
- **保存数据类型：**
 - 用户数据（金币、植物状态）
 - 环境数据
 - 植物详细状态
 - 商店数据
- **初始化加载：**
 - 启动时自动加载保存的数据
 - 首次运行初始化默认数据

2.6 异常处理

- **内存管理：**
 - 内存分配失败检测
 - 资源释放机制
- **超时处理：**
 - 植物初始化60秒超时
- **信号处理：**
 - 处理Ctrl+C终止信号
 - 处理系统终止信号

3. 系统架构

3.1 主要模块

植物养护模拟系统

- └─ 环境模拟模块
- └─ 植物生长引擎
- └─ 用户管理系统
- └─ 数据持久化模块
- └─ LVGL 图形界面
- └─ 硬件抽象层

3.2 核心数据结构

```
// 环境参数
typedef struct {
    uint8_t temperature;
    uint8_t humidity;
    uint8_t light;
    uint8_t season;
    uint32_t day;
    WeatherType weather;
} Environment;

// 植物状态
typedef struct {
    PlantType type;
    uint8_t rarity;
    GrowthStage stage;
    uint8_t health;
    uint32_t age;
    uint8_t growth;
    bool needs_water;
    bool needs_fertilizer;
    bool needs_pest_control;
    uint8_t water_count;
    uint8_t fertilize_count;
    uint8_t pest_control_count;
} PlantState;

// 用户数据
typedef struct {
    uint32_t coins;
    uint8_t plant_num;
    uint8_t plant_type[4];
} User;

// 商店数据
typedef struct {
    PlantType plant_type[4];
    uint16_t price[4];
    bool ishave[4];
} Commodity;
```

4. 非功能性需求

4.1 性能需求

- LVGL刷新频率：≥30fps
- 环境更新周期：2000ms
- 内存占用：DISP_BUF_SIZE (8004808)

4.2 可靠性需求

- 数据持久化完整性保证
- 异常状态自动恢复
- 内存泄漏防护

4.3 兼容性需求

- 支持Linux framebuffer显示
- 支持evdev输入设备
- 兼容SDL模拟环境

4.4 用户体验需求

- 植物生长状态视觉反馈
- 环境参数直观显示
- 响应式触控界面

5. 界面需求

5.1 主界面布局

```
+-----+
| [环境参数] [环境参数] [环境参数] |
| [植物1] [植物2] [植物3] [植物4] |
| [状态标签] [状态标签] ...      |
| [金币显示]                      |
+-----+
```

5.2 视觉元素

- 植物图像**：根据生长阶段变化
- 健康指示器**：颜色编码（绿→黄→红）
- 生长阶段标签**：文字描述当前阶段
- 环境参数卡片**：图标+数值显示

6. 运行环境

6.1 硬件环境

- 显示分辨率：800×480
- 输入设备：触摸屏
- 处理器：ARM Cortex-A系列或等效

6.2 软件环境

- 操作系统：Linux
- 图形库：LVGL v8+
- 依赖库：SDL（可选）

7. 测试需求

7.1 核心功能测试

- 植物生命周期完整性验证
- 环境参数影响验证
- 数据持久化一致性检查

7.2 压力测试

- 连续运行24小时稳定性
- 内存泄漏检测
- 高频用户交互响应

7.3 异常测试

- 内存分配失败处理
- 信号中断恢复
- 超时处理验证

8. 部署需求

- 提供初始化脚本
- 配置文件管理
- 数据存储路径设置
- 日志记录机制